

Email Classifier

By: Ian Flanagan
CWID: 20035092

Ian Flanagan

Have a messy inbox that makes it hard to find which emails you need?

That's where the Email Classifier comes in.

This project uses machine learning models to calculate the probability that a new, unseen email can be properly categorized based on the data of seen emails.

Upon classification, your emails will be labeled by their type (and given a confidence level) so that you'll know exactly where to find what you're looking for.



Ian Flanagan

How the data was collected

For the data collection/training process, I copy and pasted different types of emails from my inbox into .txt files in the project folder

1

Collect Email Data

Under the email directory there are different types of email categories with raw sample emails in each subdirectory.

2

Clean and Lemmatize Emails

For each email, the stopwords (“a”, “the”, “they”, etc.) were removed and the remaining words with semantic meaning were lemmatized to reduce varied forms of a word into a common base.

3

Metadata

Saved the word frequencies for each word in each email along with the total amount of emails in each category. Saved data into a JSON file for future use.

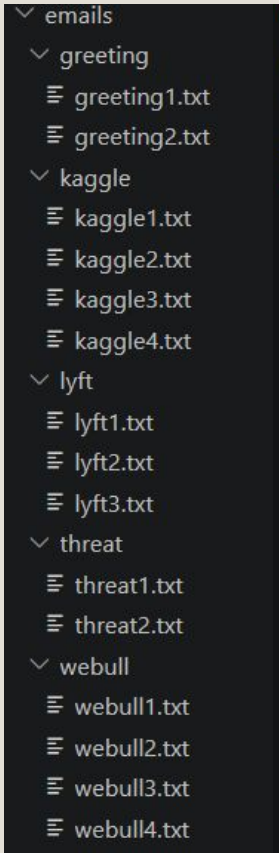
4

Ready for Classification

Now that the data has been prepared and cleaned, the algorithm can now take in a new email and give it a proper classification.

Ian Flanagan

preprocessor.py: Collecting Email Data




```python
root_folder = "emails"
stop = load_junk_words("junk.txt")
wnl = WordNetLemmatizer()
stats = handle(root_folder, wnl, stop)

save_stats(stats)
```

```python
def handle(email_dir, lemmatizer, junk_set):
    """
    Processes all categories and returns a dictionary containing
    both word frequencies and the total number of emails per category.
    """
    processed_data = {
        "metadata": {
            "category_email_counts": {},
            "total_emails": 0
        },
        "word_counts": {}
    }

    if not os.path.exists(email_dir):
        print(f"Error: Directory '{email_dir}' not found.")
        return processed_data

    for item in os.listdir(email_dir):
        item_path = os.path.join(email_dir, item)

        if os.path.isdir(item_path):
            print(f"Processing category: {item}...")

            # 1. Count the .txt files (The "Prior" data)
            email_files = [f for f in os.listdir(item_path) if f.endswith('.txt')]
            num_emails = len(email_files)
            processed_data["metadata"]["category_email_counts"][item] = num_emails
            processed_data["metadata"]["total_emails"] += num_emails

            # 2. Get the word frequencies (The "Likelihood" data)
            processed_data["word_counts"][item] = count_word_frequencies(item_path, junk_set, lemmatizer)

    return processed_data
```

1) Call the initial “handle” function to start the preprocessing phase, sending the email folder, an NLTK tool to reduce words to base form, and the list of stopwords as arguments.

2) The handle function returns the preprocessed data as a dictionary to be saved as a JSON file. The category counts and total email counts are done in this function while the individual counts per word are later determined by the count_word_frequency function.

count_word_frequnecy is sent the current email type directory, the stopwords, and the lemmatizer tool.

preprocessor.py: Clean and Lemmatize

```python
def count_word_frequencies(folder_path, junk, lemmatizer):
    """Counts frequencies for a single specific folder."""
    cnt = Counter()

    for filename in os.listdir(folder_path):
        if filename.endswith(".txt"):
            file_path = os.path.join(folder_path, filename)

            with open(file_path, 'r', encoding='utf-8') as file:
                # Read the WHOLE file as one string for better NLP context
                text = file.read()
                filt = lemma(text, lemmatizer, junk)
                cnt.update(filt)
    return cnt
```

```python
def lemma(text, lemmatizer, junk):
    """Tokenizes, tags, and lemmatizes a raw string of text."""
    # word_tokenize expects a string, not a list!
    tokens = word_tokenize(text.lower())
    tagged_tokens = pos_tag(tokens)

    final_lemmas = []
    for word, tag in tagged_tokens:
        if word.isalnum():
            w_pos = get_wordnet_pos(tag)
            lem = lemmatizer.lemmatize(word, w_pos)

            # Check if the normalized 'lem' is in our junk list
            if lem not in junk and len(lem) > 1:
                final_lemmas.append(lem)

    return final_lemmas
```

3) Each file in the folder is sent to the lemma function where it will bring all the words in the file into a base form and will then count how many times each word appears in that email type.

```python
def load_junk_words(filepath):
    """Reads the stopword file and returns a set of words."""
    junk = set()
    if os.path.exists(filepath):
        with open(filepath, 'r', encoding='utf-8') as f:
            junk = {line.strip().lower() for line in f if line.strip()}
    return junk
```

4) This is where the magic happens. Words in the file are given position tags to identify their part of speech. Once that is complete, the lemmatized words are then run against the stopwords to produce the new email.

```python
def get_wordnet_pos(treebank_tag):
    if treebank_tag.startswith('J'):
        return wordnet.ADJ
    elif treebank_tag.startswith('V'):
        return wordnet.VERB
    elif treebank_tag.startswith('N'):
        return wordnet.NOUN
    elif treebank_tag.startswith('R'):
        return wordnet.ADV
    return wordnet.NOUN
```

Ian Flanagan

nb.py and svm.py

5) When a new email is ready to be tested, it goes through the same preprocessing phase before being classified. After which the emails are split into training and testing data (80/20) to see the accuracy of the models. *JSON file not used in the models so that model accuracy could be more easily tested.

6) Along with the read of the accuracy of the models, a predictions with a given confidence level is provided for the new email to be processed.

7) Both models have high precision across email categories, but when it came to recall SVM was not as strong. Naive Bayes has a higher confidence and accuracy level but this is because of the size of the data set. Had there been thousands of emails from numerous categories, SVM would surpass NB but for acute data, NB is the winner.

```
PS C:\Users\ianfl\OneDrive\Desktop\shared\CS513\project> python nb.py
Total Emails: 87
Training set: 68 | Test set: 19

--- Model Performance Report ---
              precision    recall  f1-score   support

      invest       1.00      1.00      1.00         3
  just_juice       1.00      1.00      1.00         3
      kaggle       1.00      1.00      1.00         3
        lyft       0.80      1.00      0.89         4
       river       1.00      1.00      1.00         3
      threat       1.00      0.67      0.80         3

    accuracy                           0.95        19
   macro avg       0.97      0.94      0.95        19
weighted avg       0.96      0.95      0.95        19


Final Result: JUST_JUICE (95.56% confidence)
PS C:\Users\ianfl\OneDrive\Desktop\shared\CS513\project> python svm.py
Total Emails: 87
Training set: 68 | Test set: 19
C:\Users\ianfl\AppData\Local\Programs\Python\Python311\Lib\site-packages\s
 not be used since 'tokenizer' is not None'
  warnings.warn(

--- Accuracy Report ---
              precision    recall  f1-score   support

      invest       1.00      1.00      1.00         3
  just_juice       1.00      1.00      1.00         3
      kaggle       1.00      1.00      1.00         3
        lyft       0.80      1.00      0.89         4
       river       0.50      0.67      0.57         3
      threat       1.00      0.33      0.50         3

    accuracy                           0.84        19
   macro avg       0.88      0.83      0.83        19
weighted avg       0.88      0.84      0.83        19


Final Result: JUST_JUICE (72.79% confidence)
```

Live Demo

Thank You